

Rare Book Data Review

Richard Freedman

2025-11-15

0. Setup

Review Queue: Items Most in Need of Human Review

RareBooks_DataExtraction.ipynb flags individual extracted items with `review_flags` whenever the LLM had to guess at a garbled OCR reading, infer a field from context, or make some other judgment call. This notebook turns those flags into a single ranked **CSV review queue**: for each flagged item, what the OCR/verbatim text actually said, how the LLM resolved it into a structured field, why it was flagged, and exactly where to find it in the source PDF.

- **Source:** Structured Data/concert_program_items.json — the flat item export (one row per venue/date/organization/patron/work/performer).
- **Not used:** Structured Data/langchain_flags.csv. It's a flag log from a *different* extraction pass (code/Langchain_for_Rare_Books.ipynb, over a different, non-concert-program source) and doesn't carry a source filename per flag — only a page number — so there's no reliable way to give a “source document” for each row. If that pipeline's output ever gets a filename column, it can be folded in here the same way.

```
from pathlib import Path
import json

import pandas as pd
import plotly.express as px

data_dir = Path("Structured Data")

with open(data_dir / "concert_program_items.json", encoding="utf-8") as f:
    items = json.load(f)

print(f"{len(items)} total items")
```

```
flagged = [x for x in items if x.get("review_flags")]
print(f"{len(flagged)} items carry at least one review flag ({len(flagged) / len(items):.0%}).
```

1782 total items

335 items carry at least one review flag (19%)

1. Turn each flagged item into a review-queue row

Two judgment calls happen here:

- **LLM Resolution** — each `record_type` keeps different fields (a `work` has `composer/title/movement_or_selection`; a `performer` has `name/part_or_instrument/associated_work` ...). `resolution_text()` renders just the meaningful fields for that record type into one readable string, so the CSV can be scanned without knowing the schema.
- **Severity** — flags aren't equally urgent. A flag whose text mentions OCR garbling, illegibility, or uncertainty means the *reading itself* is in doubt; a flag like “Role standardized to ‘actor’ from ‘cast’ wording” documents a clean judgment call, not a data-quality problem. `classify_severity()` buckets flags into:
 - **High** — **OCR/legibility**: the source text may have been misread (OCR, garble, illegible, unclear, misread, uncertain, faded, unreadable, cut off)
 - **Medium** — **inference**: the text was legible but the LLM had to infer/standardize a field (inferred, assumed, implied, cross-page, best-guess, standardized, not specified, not stated, not given, missing)
 - **Low** — **informational**: a flag documenting an intentional schema choice, not a likely error

An item with multiple flags takes its *highest* severity.

```
RESOLUTION_FIELDS = {
    "venue": ["place"],
    "date": ["date_text"],
    "organization": ["name", "role"],
    "patron": ["name", "role"],
    "work": ["title", "composer", "movement_or_selection"],
    "performer": ["name", "part_or_instrument", "associated_work"],
}

def resolution_text(item):
    """Render the meaningful structured fields for this record_type as one readable string."""
    fields = RESOLUTION_FIELDS.get(item["record_type"], [])
    parts = [f"{field}: {item[field]}" for field in fields if item.get(field)]
```

```

        return " | ".join(parts)

HIGH_KEYWORDS = [
    "ocr", "garble", "illegible", "unclear", "misread", "uncertain",
    "faded", "unreadable", "cut off", "unrecoverable",
]
MEDIUM_KEYWORDS = [
    "inferred", "assumed", "implied", "cross-page", "best-guess", "best guess",
    "standardized", "not specified", "not stated", "not given", "missing", "likely",
]

def classify_severity(flags):
    """Highest severity across an item's flags: High (2) > Medium (1) > Low (0)."""
    score = 0
    for flag in flags:
        fl = flag.lower()
        if any(kw in fl for kw in HIGH_KEYWORDS):
            score = max(score, 2)
        elif any(kw in fl for kw in MEDIUM_KEYWORDS):
            score = max(score, 1)
    return score

SEVERITY_LABELS = {2: "High - OCR/legibility", 1: "Medium - inference", 0: "Low - information"}

rows = []
for item in flagged:
    score = classify_severity(item["review_flags"])
    rows.append({
        "Severity": SEVERITY_LABELS[score],
        "_severity_score": score,
        "Record Type": item["record_type"],
        "Source Document": item["filename"],
        "Page": item["page_number"],
        "Organization": item.get("manifest_organization"),
        "Date": item.get("manifest_date"),
        "OCR Original": item.get("source_text"),
        "LLM Resolution": resolution_text(item),
        "Suggested Problem": "; ".join(item["review_flags"]),
    })

review_queue = pd.DataFrame(rows).sort_values(
    ["_severity_score", "Source Document", "Page"], ascending=[False, True, True]
)

```

```
)
print(f"{len(review_queue)} rows in the review queue")
review_queue.drop(columns="_severity_score").head(15)
```

335 rows in the review queue

	Severity	Record Type	Source Document	Page	Organization	Date	O
0	High — OCR/legibility	patron	UDC20260028-10.pdf	3	Melbourne Liedertafel	1893	C
14	High — OCR/legibility	performer	UDC20260028-10.pdf	5	Melbourne Liedertafel	1893	M
10	High — OCR/legibility	performer	UDC20260028-10.pdf	16	Melbourne Liedertafel	1893	E
11	High — OCR/legibility	performer	UDC20260028-10.pdf	16	Melbourne Liedertafel	1893	E
4	High — OCR/legibility	patron	UDC20260028-10.pdf	19	Melbourne Liedertafel	1893	N
54	High — OCR/legibility	organization	UDC20260028-11.pdf	3	Melbourne Liedertafel	1893	O
55	High — OCR/legibility	organization	UDC20260028-11.pdf	4	Melbourne Liedertafel	1893	U
56	High — OCR/legibility	work	UDC20260028-11.pdf	8	Melbourne Liedertafel	1893	C
57	High — OCR/legibility	work	UDC20260028-11.pdf	8	Melbourne Liedertafel	1893	C
58	High — OCR/legibility	work	UDC20260028-11.pdf	9	Melbourne Liedertafel	1893	M
59	High — OCR/legibility	work	UDC20260028-11.pdf	11	Melbourne Liedertafel	1893	S
52	High — OCR/legibility	venue	UDC20260028-11.pdf	16	Melbourne Liedertafel	1893	ye
53	High — OCR/legibility	date	UDC20260028-11.pdf	16	Melbourne Liedertafel	1893	M
15	High — OCR/legibility	venue	UDC20260028-13.pdf	3	Melbourne Liedertafel	1889	T
32	High — OCR/legibility	performer	UDC20260028-13.pdf	4	Melbourne Liedertafel	1889	P

```
# selected document for review
review_queue[review_queue['Source Document']=='UDC20260028-21.pdf']
```

	Severity	_severity_score	Record Type	Source Document	Page	Organization
253	High — OCR/legibility	2	venue	UDC20260028-21.pdf	1	Melbourne Liedertafel
254	High — OCR/legibility	2	organization	UDC20260028-21.pdf	4	Melbourne Liedertafel
267	High — OCR/legibility	2	performer	UDC20260028-21.pdf	4	Melbourne Liedertafel
268	High — OCR/legibility	2	performer	UDC20260028-21.pdf	4	Melbourne Liedertafel
269	High — OCR/legibility	2	performer	UDC20260028-21.pdf	4	Melbourne Liedertafel
...	...	...	...	...	...	...
260	Medium — inference	1	work	UDC20260028-21.pdf	14	Melbourne Liedertafel
262	Medium — inference	1	work	UDC20260028-21.pdf	14	Melbourne Liedertafel
263	Medium — inference	1	work	UDC20260028-21.pdf	14	Melbourne Liedertafel
266	Low — informational	0	performer	UDC20260028-21.pdf	9	Melbourne Liedertafel
255	Low — informational	0	patron	UDC20260028-21.pdf	18	Melbourne Liedertafel

2. Quick shape check before exporting

```
fig = px.bar(
    review_queue.groupby(["Severity", "Record Type"]).size().reset_index(name="count"),
    x="Severity",
    y="count",
    color="Record Type",
    title="Flagged items by severity and record type",
    category_orders={"Severity": ["High - OCR/legibility", "Medium - inference", "Low - info"]},
)
fig.update_layout(height=500)
fig.show()
```

Unable to display output for mime type(s): application/vnd.plotly.v1+json

```
fig = px.bar(
    review_queue.groupby("Organization").size().reset_index(name="flagged_items").sort_values("flagged_items", ascending=False),
    x="flagged_items",
    y="Organization",
    orientation="h",
    title="Flagged items by organization",
)
fig.update_layout(yaxis={"categoryorder": "total ascending"}, height=400)
fig.show()
```

Unable to display output for mime type(s): application/vnd.plotly.v1+json

3. Export the CSV

Sorted worst-first (High severity → Medium → Low, then by document/page) so a human reviewer can work down the list and stop whenever they've covered the material that matters most.

```
out_path = data_dir / "review_queue.csv"
review_queue.drop(columns="_severity_score").to_csv(out_path, index=False)
print(f"Wrote {len(review_queue)} rows to {out_path}")
```

Wrote 335 rows to Structured Data/review_queue.csv